

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS:

Duration :60 Mins
On class
Total Marks:50

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I Prartana T(192421099) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Prartana T

Annexure A

ERROR ANALYSIS TASK

Q1. The following C program is intended to perform string processing, function calls, array manipulation, conditional execution, pointer operations, switch-case selection, and loop execution. However, the program contains several errors introduced during different phases of compilation, including lexical, syntax, semantic, logical, and runtime errors.

Given Program

```
#include <stdio.h>
#include <string.h>

int findLength(char str[])
{
    return strlen(str);
}

int main()
{
    char name[20] = "Compiler"

    int count;

    count = findLength();

    if(count > 5
    {
        printf("Length is greater than 5\n");
    }

    int marks[4] = {85, 90, 75, 80};

    printf("%d\n", marks[4]);

    int choice = 2;

    switch(choice)
    {
        case 1:
            printf("Option 1\n");
            break;

        case 2
            printf("Option 2\n");
            break;

        default:
            printf("Invalid Choice\n");
```

```

    }

    int *ptr;
    printf("%d\n", *ptr);

    int total = 100;
    int students = 0;

    float average = total / students;

    int i = 1;

    while(i <= 5)
    {
        printf("%d\n", i);
    }

    return 0;
}

```

Tasks

- Identify all errors in the program. Mention the line number, error type, and reason for each error. (5 Marks)
- Classify the identified errors as Lexical, Syntax, Semantic, Logical, or Runtime errors in a suitable table. (5 Marks)
- Explain at which phase of compilation each category of error is detected. Also mention which errors are detected only during execution. (4 Marks)
- Rewrite the program after correcting all the errors so that it executes successfully. (4 Marks)
- Suggest any three compiler features or warnings that can help detect such programming errors before execution. (2 Marks)

Q2. The following C program is intended to perform function calls, array processing, conditional execution, switch-case selection, and loop operations. However, several syntax errors have been intentionally introduced by violating the grammatical rules of the C language. These errors include missing delimiters, unmatched parentheses, incorrect control structures, malformed function definitions, and incomplete statements. Such errors are normally detected during the syntax analysis (parsing) phase of a compiler.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>
```

```

int maximum(int x, int y
{
    if(x > y)
        return x;
    else
        return y;
}

int main()
{
    int marks[5] = {70, 80, 90, 85, 95};

    int max = maximum(marks[0], marks[1]);

    switch(max)
    {
        case 80:
            printf("Maximum is 80\n")
            break;

        case 90
            printf("Maximum is 90\n");
            break;

        default:
            printf("Other Value\n");
    }

    do
    {
        printf("%d\n", max);
        max--;
    } while(max > 75);

    if(max == 75)
        printf("Limit Reached\n")
    }

    return 0;

```

Tasks

- (a) Identify all syntax errors in the program. Mention the line number and reason for each error. (4 Marks)
- (b) Classify the identified syntax errors under the appropriate categories and present them in a table. (3 Marks)
- (c) Correct the syntax errors and rewrite the complete program. (4 Marks)
- (d) Explain how the parser detects syntax errors and how error recovery enables further parsing. (2 Marks)
- (e) Suggest any two improvements that can help a compiler generate better syntax error messages. (2 Marks)

Q3. The following C program is intended to perform factorial computation, function calls, array processing, pointer manipulation, conditional execution, switch-case selection, and loop operations. However, the program contains several errors introduced during different phases of compilation and execution, including lexical, syntax, semantic, intermediate code generation, code optimization, and runtime errors.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>
```

```
int factorial(int n)
```

```
{
```

```
    if(n <= 1)
```

```
        return 1;
```

```
    return n * factorial(n - 1);
```

```
}
```

```
int main()
```

```
{
```

```
    int @num = 5;
```

```
    int value = 10
```

```
    int result;
```

```
    result = factorial();
```

```

if(result >= 100)
{
    printf("Large Number\n");
}

switch(result)
{
    case 24:
        printf("Factorial of 4\n");
        break;

    case 120
        printf("Factorial of 5\n");
        break;

    default:
        printf("Other Value\n");
}

int numbers[5] = {2,4,6,8,10};
printf("%d\n", numbers[5]);

int *ptr;
printf("%d\n", *ptr);

int a = 50;
int b = 0;
int c = a / b;

int sum = result + 0 + value;

int i = 1;
while(i <= 5)
{
    printf("%d\n", i);
}

return 0;
}

```

Tasks

- (a) Identify all the errors in the program. Mention the line number, error type, and reason for each error. (4 Marks)
- (b) Classify the identified errors according to the appropriate compiler phase (Lexical, Syntax, Semantic, Intermediate Code Generation, Code Optimization, and Runtime) using a suitable table. (4 Marks)
- (c) Explain how the compiler processes the program by identifying the errors detected during each compilation phase and those detected only at runtime. (3 Marks)
- (d) Rewrite the complete corrected program so that it compiles and executes without errors. (2 Marks)
- (e) Suggest any two improvements in compiler design or optimization techniques that could detect or prevent such errors before program execution. (2 Marks)

Solutions

Question 1

- (a) Identify all errors in the program

1. Line 35: Missing semicolon (;) after char name[20] = "Compiler" – Syntax Error.
2. Line 37: findLength() is called without passing the required argument (name) – Semantic Error.
3. Line 38: Missing closing parenthesis) in the if statement – Syntax Error.
4. Line 43: marks[4] accesses an invalid array index. Valid indices are 0–3 – Runtime Error.
5. Line 50: Missing colon (:) after case 2 – Syntax Error.
6. Line 58: Pointer ptr is not initialized before dereferencing (*ptr) – Runtime Error.
7. Line 61: Division by zero because students = 0 – Runtime Error.
8. Lines 63–66: Variable i is never incremented inside the while loop, causing an infinite loop – Logical Error.

- (b) Classify the identified errors

Lexical Errors

- No lexical errors are present.

Syntax Errors

- Missing semicolon after `char name[20] = "Compiler".`
- Missing closing parenthesis in the if statement.
- Missing colon after case 2.

Semantic Errors

- Calling `findLength()` without passing the required argument.

Logical Errors

- Infinite loop because `i` is never incremented.

Runtime Errors

- Accessing array element `marks[4]`.
- Dereferencing an uninitialized pointer.
- Division by zero.

(c) Explain at which phase each error is detected

1. Lexical Analysis Phase
 1. Detects invalid characters or illegal tokens.
 2. No lexical errors are present in this program.
2. Syntax Analysis (Parser) Phase
 1. Detects missing semicolons, parentheses, colons, and other grammar-related errors.
3. Semantic Analysis Phase
 1. Detects incorrect function calls, type mismatches, undeclared variables, etc.
 2. Example: Calling `findLength()` without an argument.
4. Logical Errors
 1. Not detected by the compiler.
 2. Found only by testing or debugging because the program compiles but gives incorrect behavior.
5. Runtime Errors
 1. Detected only during program execution.
 2. Examples:
 1. Array index out of bounds.
 2. Dereferencing an uninitialized pointer.

3. Division by zero.

(d) Corrected Program

```
#include <stdio.h>#include <string.h>
int findLength(char str[])
{
    return strlen(str);
}
int main()
{
    char name[20] = "Compiler";
    int count;

    count = findLength(name);

    if(count > 5)
    {
        printf("Length is greater than 5\n");
    }

    int marks[4] = {85, 90, 75, 80};
    printf("%d\n", marks[3]);

    int choice = 2;

    switch(choice)
    {
        case 1:
            printf("Option 1\n");
            break;

        case 2:
            printf("Option 2\n");
            break;

        default:
            printf("Invalid Choice\n");
    }

    int value = 10;
    int *ptr = &value;
    printf("%d\n", *ptr);

    int total = 100;
    int students = 5;
    float average = (float)total / students;
    printf("Average = %.2f\n", average);

    int i = 1;
```

```

while(i <= 5)
{
    printf("%d\n", i);
    i++;
}

return 0;
}

```

Output:

The screenshot shows a code editor with C code on the left and its output on the right. The code includes a switch statement for menu options, a pointer example, and a calculation of an average. The output shows the results of these operations, including a successful code execution message.

```

24  int choice = 2;
25
26  switch(choice)
27  {
28      case 1:
29          printf("Option 1\n");
30          break;
31
32      case 2:
33          printf("Option 2\n");
34          break;
35
36      default:
37          printf("Invalid Choice\n");
38  }
39
40  int value = 10;
41  int *ptr = &value;
42  printf("%d\n", *ptr);
43
44  int total = 100;
45  int students = 5;
46  float average = (float)total / students;
47  printf("Average = %.2f\n", average);
48
49  int i = 1;

```

Output:

```

Length is greater than 5
80
Option 2
10
Average = 20.00
1
2
3
4
5
=== Code Execution Successful ===

```

(e) Suggest any three compiler features or warnings

1. Syntax Checking – Detects syntax errors such as missing semicolons, parentheses, and colons during parsing.
2. Compiler Warnings (-Wall and -Wextra) – Warns about uninitialized variables, incorrect function calls, unreachable code, and other potential issues.
3. Static Code Analysis Tools – Tools like Cppcheck and Clang-Tidy detect possible runtime errors such as null pointers, array out-of-bounds access, and division by zero before execution.

Question 2

(a) Identify all syntax errors in the program

1. Line 88: Missing closing parenthesis) in the function declaration `int maximum(int x, int y)` – Syntax Error.
2. Line 103: Missing semicolon (;) after `printf("Maximum is 80\n")` – Syntax Error.
3. Line 105: Missing colon (:) after case 90 – Syntax Error.
4. Line 115: Incorrect placement of `while(max > 75);` in the do-while loop due to improper block structure – Syntax Error.
5. Line 117: Missing closing parenthesis) in `printf("Limit Reached\n")` – Syntax Error.
6. Line 118: Missing closing brace } for the if block and/or main() function – Syntax Error.

(b) Classify the identified syntax errors

Missing Delimiters

- Missing semicolon after `printf("Maximum is 80\n")`.
- Missing colon after case 90.

Unmatched Parentheses

- Missing) in the function declaration.
- Missing) in `printf("Limit Reached\n")`.

Incorrect Control Structure

- Improper do-while loop structure.

Incomplete Statements / Missing Braces

- Missing closing brace } at the end of the program.

(c) Corrected program

```
#include <stdio.h>
int maximum(int x, int y)
{
    if(x > y)
        return x;
    else
        return y;
}
```

```
}
int main()
{
    int marks[5] = {70, 80, 90, 85, 95};
    int max = maximum(marks[0], marks[1]);

    switch(max)
    {
        case 80:
            printf("Maximum is 80\n");
            break;

        case 90:
            printf("Maximum is 90\n");
            break;

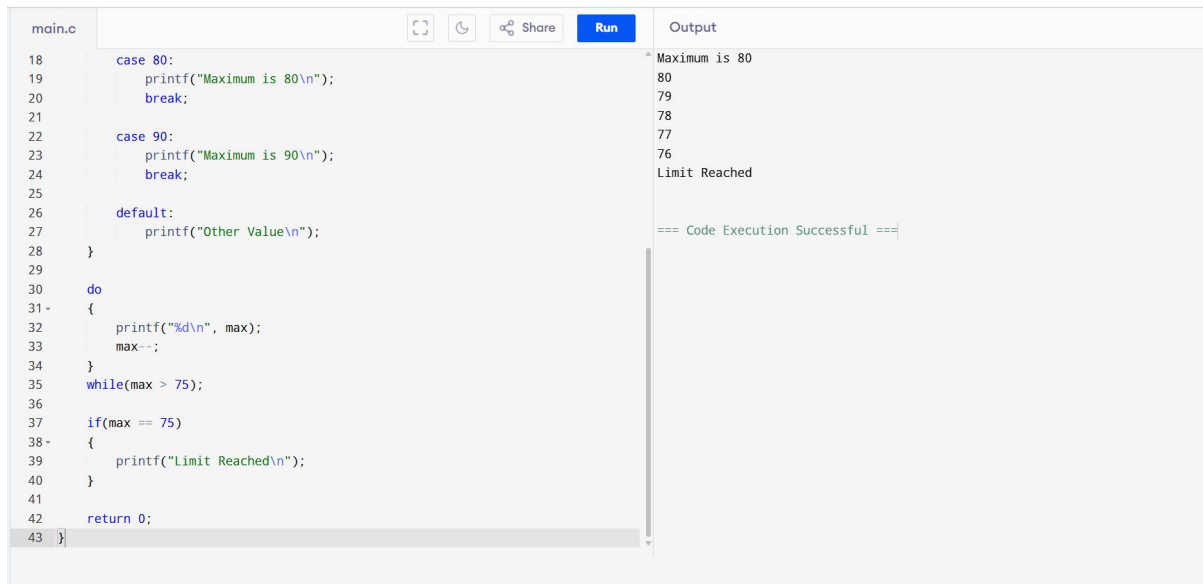
        default:
            printf("Other Value\n");
    }

    do
    {
        printf("%d\n", max);
        max--;
    }
    while(max > 75);

    if(max == 75)
    {
        printf("Limit Reached\n");
    }

    return 0;
}
```

Output



```
main.c
18 case 80:
19     printf("Maximum is 80\n");
20     break;
21
22 case 90:
23     printf("Maximum is 90\n");
24     break;
25
26 default:
27     printf("Other Value\n");
28 }
29
30 do
31 {
32     printf("%d\n", max);
33     max--;
34 }
35 while(max > 75);
36
37 if(max == 75)
38 {
39     printf("Limit Reached\n");
40 }
41
42 return 0;
43 }
```

Output

```
Maximum is 80
80
79
78
77
76
Limit Reached

=== Code Execution Successful ===
```

(d) Explain how the parser detects syntax errors and how error recovery enables further parsing

1. The parser checks whether the program follows the grammar rules of the C language.
2. It detects errors such as missing semicolons, parentheses, braces, colons, and incorrect control structures.
3. When a syntax error is found, the parser reports the error with its location.
4. Using error recovery techniques (such as panic mode or phrase-level recovery), the parser skips the incorrect part and continues parsing the remaining program so that multiple syntax errors can be identified in a single compilation.

(e) Suggest any two improvements that can help a compiler generate better syntax error messages

1. Provide clear error messages with line numbers and suggestions, such as indicating the missing symbol (;,), or }).
2. Use intelligent error recovery techniques so the compiler can continue parsing after an error and report multiple syntax errors instead of stopping at the first one.

Question 3

(a) Identify all the errors in the program

1. Line 147: `int @num = 5;` contains an invalid character (@) in the variable name – Lexical Error.

2. Line 148: Missing semicolon (;) after int value = 10 – Syntax Error.
3. Line 150: factorial() is called without the required argument – Semantic Error.
4. Line 161: Missing colon (:) after case 120 – Syntax Error.
5. Line 168: numbers[5] accesses an invalid array index. Valid indices are 0–4 – Runtime Error.
6. Line 170: Pointer ptr is used without initialization – Runtime Error.
7. Line 173: Division by zero (a / b, where b = 0) – Runtime Error.
8. Line 174: result + 0 + value contains an unnecessary addition with 0 – Code Optimization Issue.
9. Lines 175–178: Variable i is never incremented inside the while loop, causing an infinite loop – Runtime/Logical Error.

(b) Classify the identified errors according to compiler phase

Lexical Errors

- Invalid character (@) in the variable name.

Syntax Errors

- Missing semicolon after int value = 10.
- Missing colon after case 120.

Semantic Errors

- Calling factorial() without passing the required argument.

Intermediate Code Generation Issues

- No major intermediate code generation errors are present. The compiler cannot generate proper intermediate code until syntax and semantic errors are corrected.

Code Optimization Issues

- Expression result + 0 + value contains redundant addition with 0, which can be optimized to result + value.

Runtime Errors

- Accessing numbers[5] (array index out of bounds).
- Dereferencing an uninitialized pointer.
- Division by zero.
- Infinite loop because i is never incremented.

(c) Explain how the compiler processes the program

1. During Lexical Analysis, the compiler detects invalid tokens such as the character @ in the variable name.
2. During Syntax Analysis, it detects grammar errors like missing semicolons and missing colons in the switch statement.
3. During Semantic Analysis, it detects incorrect function calls, such as calling factorial() without passing the required argument.
4. During Intermediate Code Generation, the compiler converts the valid program into intermediate code. This phase cannot proceed correctly if syntax or semantic errors exist.
5. During Code Optimization, redundant operations such as adding 0 (result + 0 + value) are removed to improve efficiency.
6. During Program Execution (Runtime), errors such as array index out of bounds, division by zero, uninitialized pointer access, and infinite loops are detected.

(d) Corrected Program

```
#include <stdio.h>
int factorial(int n)
{
    if(n <= 1)
        return 1;
    return n * factorial(n - 1);
}
int main()
{
    int num = 5;
    int value = 10;
    int result;

    result = factorial(num);

    if(result >= 100)
    {
        printf("Large Number\n");
    }

    switch(result)
    {
        case 24:
            printf("Factorial of 4\n");
            break;

        case 120:
            printf("Factorial of 5\n");
            break;

        default:
```

```

        printf("Other Value\n");
    }

    int numbers[5] = {2,4,6,8,10};
    printf("%d\n", numbers[4]);

    int x = 20;
    int *ptr = &x;
    printf("%d\n", *ptr);

    int a = 50;
    int b = 5;
    int c = a / b;

    int sum = result + value;

    int i = 1;
    while(i <= 5)
    {
        printf("%d\n", i);
        i++;
    }

    return 0;
}

```

Output

```

33     default:
34         printf("Other Value\n");
35     }
36
37     int numbers[5] = {2,4,6,8,10};
38     printf("%d\n", numbers[4]);
39
40     int x = 20;
41     int *ptr = &x;
42     printf("%d\n", *ptr);
43
44     int a = 50;
45     int b = 5;
46     int c = a / b;
47
48     int sum = result + value;
49
50     int i = 1;
51     while(i <= 5)
52     {
53         printf("%d\n", i);
54         i++;
55     }
56
57     return 0;
58 }

```

```

* Large Number
Factorial of 5
10
20
1
2
3
4
5

=== Code Execution Successful ===

```

(e) Suggest any two improvements in compiler design or optimization techniques

1. Advanced Static Code Analysis

- The compiler can detect possible runtime errors such as null pointers, array out-of-bounds access, and division by zero before execution.

2. Better Code Optimization

- The compiler can eliminate redundant operations (such as adding 0), remove unnecessary code, and optimize loops to improve execution speed and memory usage.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand